

John Louie M. Sude

Caloocan City, NCR | sudejohnlouie@gmail.com | +63 947 316 1730 | linkedin.com/in/jlsude
github.com/jlsude | jlsude.web.app

Education

Bachelor of Science in Computer Engineering Major in Data Science2021- 2025

Technological Institute of the Philippines - Manila (Magna Cum Laude)

Technical Skills

- Languages
 - HTML/CSS, Python, JavaScript/TypeScript, Dart, SQL
- Technologies
 - Backend: Node.js, Express
 - Frontend: Flutter, React.js, Next.js, TailwindCSS
 - Database: Firebase Firestore, PostgreSQL, Supabase
 - Blockchain & Payments: Solidity, Xendit
 - AI/ML: Tensorflow, Scikit-learn, Pandas, Gemma
- Development Tools
 - Firebase, Supabase, GitLab, GitHub, Postman, Figma

Work Experience

PGX Group Inc. May 2025 - Apr 2026

Full Stack Developer (Contract)

Flutter, Node.js, Firebase Suite, Xendit, Ethereum (SKALE), Gemma

- Designed and implemented an **idempotent wallet system** with a **double-entry accounting ledger** and auditable transaction log, managing 100% of user balances and processing significant transaction volume through Xendit during early access launch.
- Architected 100+ **RESTful APIs** using **Node.js** and **Express** on **Firebase Cloud Functions** for an event ticketing platform. Implemented a granular event-scoped **RBAC** system with organizer-defined roles modeled as Firestore subcollections with denormalized role data on event team member documents for efficient single-read authorization, enforcing all access control server-side beyond Firebase's client-side security rules.
- Designed a cost-efficient **Firestore** data model with intentional **denormalization** across tickets and events to prevent read amplification. Implemented **Firestore security rules** for role-scoped real-time reads, eliminating the need for websocket infrastructure while maintaining granular access control on the client.
- Developed a secure QR code ticket validation system using **AES-256-GCM encryption** with time-bound payloads expiring after 60 seconds to prevent screenshot and replay attacks. Enforced backend-only verification with **Firestore transactions** to eliminate race conditions during concurrent scans, restricted to authorized scanners via the platform's **RBAC system**.
- Developed and deployed a custom **ERC-721 smart contract** on the **SKALE network** for gasless NFT-based ticketing, integrating on-chain minting and transfers via **ethers.js** triggered by Firestore document events. Enabled seamless blockchain ticketing without exposing users to gas fees or wallet complexity.
- Integrated a **Gemma 3 LLM** chatbot for event organizers to query real-time ticket sales analytics, implementing **conversation context management** and an **aggregated sales cache** with staleness validation to minimize Firestore read operations on every prompt.
- Configured **GitLab CI/CD** pipelines with multi-environment deployments across staging and production for both frontend and backend services.
- Implemented an automated test suite using **Jest** with 67 tests covering critical financial logic including ticket sales calculations and event earnings disbursements.
- Developed and maintained the **Flutter** web frontend with mobile responsiveness as the sole developer, implementing **Riverpod** state management and real-time Firestore listeners integrated with the platform's backend services.

Projects

Heart Attack Risk Assessment ML Web ApplicationJan 2025

React.js, TypeScript, ONNX.js, Scikit-learn, Pandas, Plotly Express, Firebase

heart-attack-prediction-with-svm.web.app/

- Trained and optimized an **SVM classifier** for heart attack risk prediction, improving validation accuracy from **78% to 90.5%** through **GridSearchCV** hyperparameter tuning.
- Exported the trained model to **ONNX format** and deployed it for client-side inference via **ONNX.js**, eliminating the need for a backend inference server.
- Built a responsive clinical input interface in **React.js** for healthcare professionals to assess patient risk in real time.

Mammal Species Classification with CNN and Regularization — Kaggle NotebookJan 2025

Tensorflow, Matplotlib

www.kaggle.com/code/cmosbattery/mammal-classification-with-cnn-and-regularization

- Designed and trained a **Convolutional Neural Network** from scratch using **TensorFlow's Sequential API** on a dataset of 9,000 images spanning 45 mammal classes.
- Implemented **Data Augmentation**, **Dropout**, and **L2 Regularization** to mitigate overfitting and improve model generalization.